

Data Mixture Inference: What do BPE Tokenizers Reveal about their Training Data?

Authors: Jonathan Hayase, Alisa Liu, Yejin Choi, Sewoong Oh, Noah A. Smith

Year: 2024 · Venue: arXiv (Cornell University) · Difficulty: ●●●○○ (Advanced)

[10.48550/arxiv.2407.16607](https://arxiv.org/abs/10.48550/arxiv.2407.16607)

TL;DR

The paper shows that you can estimate what mix of languages, code, and data sources a language model's tokenizer was trained on just by reading its BPE merge list, and they turn that idea into a linear program (an optimization problem with linear constraints) that recovers mixture proportions surprisingly well.

Prerequisites

- BPE tokenizer (a text-splitting method that starts from small pieces like bytes or characters and repeatedly merges the most common adjacent pair)
- Token frequency (how often a token or token pair appears in data)
- Linear program (an optimization problem where both the objective and constraints are linear equations or inequalities)
- Mixture distribution (a weighted combination of several data sources, like 70% English and 30% Python)
- Inference attack (a method for extracting hidden information about a model or its training data)

The Big Idea

This paper is about a simple but sharp idea: tokenizers leak information. We usually think the secret sauce of a large language model is hidden in the model weights or the training corpus. These authors point out that the tokenizer itself, especially a BPE tokenizer (a tokenizer that builds its vocabulary by repeatedly merging the most common adjacent symbol pair), already reveals a lot about the data it saw. That is the whole punchline. If a tokenizer learned to merge things that are common in code very early, that is a fingerprint of code-heavy training data. If it learned lots of merges useful for non-English text, that points to multilingual data.

What makes this paper interesting is that it turns that intuition into an actual attack. Not a vague "maybe we can tell" story. They use the ordered list of BPE merges and a few example datasets for candidate categories, then solve for the mixture weights that best explain the observed merge order. In plain English: if we know what English text looks like, what Python looks like, what French looks like, and so on, we can ask, "What combination of these would make this exact merge list make sense?" They formulate that as a linear program, which is a clean and surprisingly elegant move.

Why should you care? Because pretraining data is mostly secret, and that secrecy matters. It affects bias, multilingual ability, copyright risk, and what the model is actually good at. Companies may release a tokenizer even when they do not release the data. This paper says that doing so is not harmless. The tokenizer is like seeing the wear pattern on a chef's knife: even if you never saw the kitchen, you can

infer what got chopped most often.

The results are also juicy. They claim GPT-4o and Mistral NeMo tokenizers are much more multilingual than earlier versions. They infer that GPT-3.5 and Claude tokenizers are heavily code-oriented, around 60% code. And they argue Llama 3's tokenizer extension is mostly about multilingual coverage. Some of these findings confirm public claims. Others go beyond what companies disclosed. That makes the paper feel less like a toy exercise and more like a real auditing tool.

The Problem

The problem is data mixture inference (recovering the proportions of different categories in training data). Suppose the hidden tokenizer training set is made from k categories, like English, French, Python, and web text sources, with unknown weights $w_1, \dots, w_k$ where $w_i \geq 0$ and $\sum_i w_i = 1$. The goal is to estimate those w_i values. This is different from membership inference (guessing whether one exact example was in the training set). Here we want a global property of the dataset, not a yes/no answer about one document.

Why is this hard? Because the training data is hidden, and most previous attacks look at the model itself. That usually means probing probabilities, memorization, or training many shadow models (copycat models trained to imitate the target setup) and then learning a meta-classifier (a classifier that predicts dataset properties from model behavior). That is expensive, indirect, and often weak. For language models in particular, membership inference has been pretty brittle, especially when the data is deduplicated. So the usual toolbox is not great for answering, "How much code was in pretraining?" or "How multilingual was this model really?"

Their key observation is that BPE training creates a long chain of frequency-based decisions. At each step, BPE picks the most frequent adjacent token pair in the current corpus and merges it. So if pair a was merged before pair b , then at that moment a had to be at least as frequent as b . That gives you inequality constraints. If category i contributes expected pair frequency $f_i(a)$ for pair a , then under mixture weights w_i , the total frequency is roughly $\sum_i w_i f_i(a)$. The merge order implies many constraints of the form

$$\sum_i w_i f_i(a) \geq \sum_i w_i f_i(b).$$

Stack enough of these together, and you can solve for the w_i . That is the clever part. They are extracting hidden dataset proportions from the ranking of pair frequencies, not from the model's outputs.

There is also a practical reason this works better than it sounds. BPE merge order is extremely sensitive to data composition. A tiny change in what text dominates the corpus can move some merges much earlier or later. The paper leans into that sensitivity instead of treating it as noise. In effect, the tokenizer is a compressed record of what patterns were common in the training data. The authors exploit that record.

Key Contributions

- They define the task of data mixture inference for language model pretraining data — estimating category proportions rather than checking whether one example was included. This matters because many real questions about model behavior are about dataset composition, not membership.
- They identify BPE merge lists as a new leakage channel. That is the paper's main conceptual contribution, and it is genuinely clever because tokenizers are public and usually treated as harmless

infrastructure.

- They formulate the attack as a linear program (an optimization problem with linear constraints) using inequalities implied by merge order. This is cleaner and more interpretable than training a black-box meta-classifier.
- They show strong recovery in controlled settings across natural languages, programming languages, and data sources. So the method is not just a cute anecdote about one tokenizer; it works across several mixture types.
- They apply the method to real released tokenizers and recover both known and new properties, including substantial multilinguality in GPT-4o and Mistral NeMo and heavy code presence in GPT-3.5 and Claude tokenizers. This is what makes the work feel consequential.
- They argue that tokenizer release can enable external auditing of hidden pretraining choices, including safety, bias, and copyright-relevant properties. That broader implication is bigger than the specific attack.

Where This Fits

This paper sits between two lines of work. One is attacks on language models, especially membership inference (guessing whether a specific example was in training) and memorization extraction (trying to pull memorized text out of the model). The other is distribution inference (recovering broad properties of training data), which has been studied more in classifiers, convolutional neural networks, and generative adversarial networks than in language model pretraining. The authors' move is to bring distribution inference into the LM world, but through the tokenizer rather than the model weights or outputs.

The idea also builds directly on how BPE works. BPE is old and widely used, but people mostly treat it as an engineering choice for compression and token efficiency. This paper says: no, the merge list is evidence. That is the lineage of ideas here. Take a standard algorithm, notice that its internal decisions are frequency-ranked, and reinterpret those decisions as a side channel (an unintended information leak). In that sense, the paper competes less with fancy attack models and more with the assumption that tokenizers are innocuous. I think that is why the paper stands out: the core trick feels obvious only after someone points it out.

How It Works

First, the simple version: they treat the tokenizer's merge list like a fingerprint of the data it saw during training. In byte-pair encoding, or BPE (a tokenizer that repeatedly merges the most common neighboring token pair), the t -th merge must have been the most frequent pair at step t . That one fact gives a lot away. If you have sample corpora for candidate categories — say English, French, Python, books, web, Wikipedia — you can ask: what mixture of these categories would make the observed merge sequence make sense? Their whole method is built around that question.

Step by step, here is what they do. Suppose there are n categories with unknown mixture weights $\alpha_1, \dots, \alpha_n$, where $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$. For each category i , they take a sample corpus and simulate the target tokenizer's merge process on it. After the first $t - 1$ merges have been applied, they count how often every token pair p appears. Call that count $c_{i,p}^{(t)}$. If the real tokenizer chose merge $m^{(t)}$ at step t , then under the true training mixture, that pair should beat every competitor. So they write one linear inequality per competing pair: the weighted count of the true merge must be at least as large as the

weighted count of any other pair. This turns the reverse-engineering problem into a linear program, or LP (an optimization problem with linear constraints and a linear objective).

Now, the full picture: real data is noisy. Their sample corpora are not the exact tokenizer training set, so the constraints will not all fit perfectly. To avoid the LP becoming impossible to solve, they add slack variables, which are just nonnegative “forgiveness terms” that allow some constraints to be violated. There is one slack $v^{(t)}$ for each merge step and one slack v_p for each competing pair. Then they minimize the total violation. Intuitively, they are asking for the mixture α that makes the observed merge list look as consistent as possible with the category corpora, while paying a penalty whenever the counts disagree. The loss is not learned with gradient descent. It is solved exactly by an LP solver.

The clever engineering bit is scale. A naive version is enormous. If the vocabulary size is V , there are about $O(V)$ merge steps and $O(V^2)$ possible competing pairs, so the full constraint set blows up to about $O(V^3)$. That is dead on arrival. They cut this down in three ways. First, they only use the first T merges, with values like $T = 30,000$ in practice. Second, they store pair counts incrementally, because most pair counts do not change from one merge step to the next. Only pairs that overlap the latest merge need updating, and that is usually just a few hundred changes. Third, they use lazy constraint and variable generation: start with a tiny LP, solve it, find which omitted constraints are violated, add only those, and repeat. It is like debugging only the lines that actually fail instead of rereading the whole codebase every time.

Inference is straightforward once all that machinery is in place. They initialize with uniform mixture weights and zero slacks, run a fast violation checker over the first T merges, build a reduced LP from the violated constraints, solve it, and iterate until no unseen constraint is violated. The final $\hat{\alpha}$ is their estimate of the tokenizer’s training mixture. In controlled experiments, tokenizers are trained on 10 GB mixtures with vocabulary size 30,000, and the attack estimates pair frequencies using 1 GB per category. Runtime is practical but not cheap: easy synthetic cases solve in minutes, while hard commercial tokenizer cases can take up to a day. That sounds heavy, but the payoff is strong: they recover training mixtures far more accurately than simple heuristics based on compression efficiency or looking at the vocabulary.

The merge list is a witness

A BPE merge list is not just a bag of tokens. It is an ordered record of which pair was most common at each training step. That order gives much stronger clues about the training data than the final vocabulary alone.

Turn reverse engineering into an LP

The core move is simple and sharp: each merge step gives linear inequalities in the unknown mixture weights. Once you write those down, the whole problem becomes a linear program. That is clever because LP solvers are mature, exact, and much easier to trust than a hand-tuned neural estimator.

Slack variables save the method

Without slack, the constraints would often be inconsistent because the attacker only has sampled corpora, not the exact tokenizer training set. The slack variables act like shock absorbers. They let the method survive sampling noise while still preferring mixtures that explain the merge list tightly.

Only update what changed

Most pair counts stay the same after a merge. The paper exploits that hard. By storing only incremental count changes and generating constraints lazily, they turn an impossible $O(V^3)$ monster into something you can actually run.

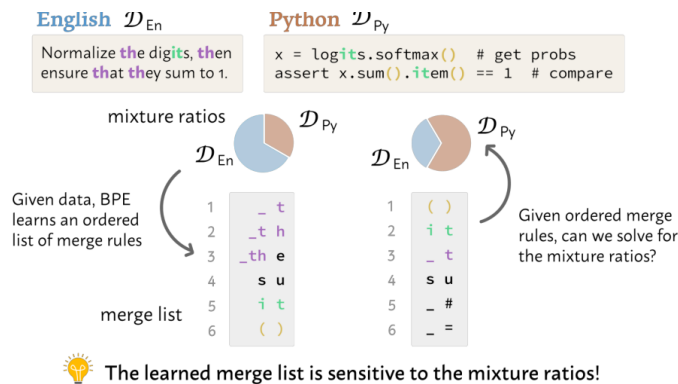


Figure 1: **Illustration of our problem statement** on a simple example where two tokenizers are trained on different mixtures of English and Python data. During training, the BPE algorithm iteratively finds the pair of tokens with the highest frequency in the training data, adds it to the merge list, then applies it to the dataset before finding the next highest-frequency pair. To encode text at inference time, the learned merge rules are applied in order. The resulting order of merge rules is extremely sensitive to the proportion of different data categories present. Our goal is to solve for these proportions, a task which we call *data mixture inference*.

This is the intuition picture. Two tokenizers trained on different English/Python mixtures produce different early merges, because common pairs in prose and common pairs in code are just not the same. The point is simple but important: the merge list is not random bookkeeping. It leaks the training mixture.



Figure 3: **Illustration of our method** on a simple example. We know that after applying in the first $t - 1$ merges to the training data, the t^{th} merge must be the most common pair. More explicitly, this means that α_i should give a vector in which the value corresponding to the true next merge is the maximum. Our attack collects these inequalities at every time step to construct the linear program.³

This figure explains the LP visually. At each merge step, the true next merge must sit at the top of the weighted pair-count vector. So every step gives a set of inequalities, and stacking those inequalities across time pins down the mixture weights.

The Math

Let there be n candidate data categories. Category i has distribution D_i , and the unknown tokenizer training mixture is

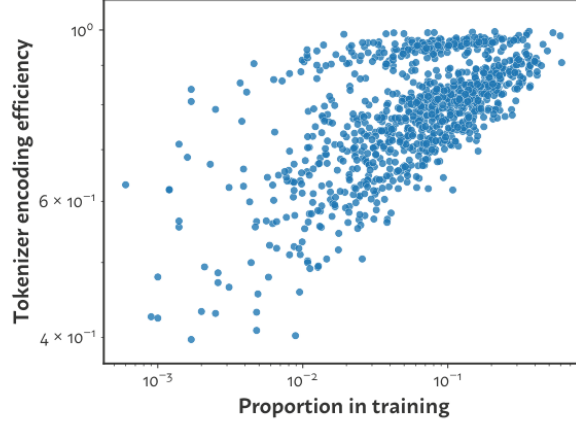


Figure 5: **Relationship between a language’s proportion in training and the resulting tokenizer’s encoding efficiency on that language**, shown for mixtures of $n = 10$ languages. The encoding efficiency is defined as the byte-to-token ratio of a given tokenizer on a given language, normalized by that of a tokenizer trained *only* on that language. While more training data leads to better encoding efficiency, the correlation is not strong enough to recover a prediction nearly as precise as our attack. A baseline based on this relationship achieves $\log_{10}$ MSE of -2.22 , compared to our attack’s -7.66 .

This figure quietly explains why the tokenizer-efficiency baseline is weak. Encoding efficiency does correlate with training proportion, but the relationship is noisy and indirect. The LP method wins because it uses the actual BPE training rule, not a rough side effect of it.

$$P = \sum_{i=1}^n \alpha_i^* D_i, \quad \alpha_i^* \geq 0, \quad \sum_{i=1}^n \alpha_i^* = 1. \quad (1)$$

In plain English: the tokenizer was trained on a weighted blend of categories, and the whole goal is to recover those weights α_i^* . The weights are proportions, so they must be nonnegative and add up to 1.

The tokenizer exposes an ordered merge list

$$m^{(1)}, m^{(2)}, \dots, m^{(M)}, \quad (2)$$

where $m^{(t)}$ is the pair merged at step t . For each category i , after applying the first $t - 1$ merges to its sample corpus, define $c_{i,p}^{(t)}$ as the count of pair p at that stage.

$$c_{i,p}^{(t)} = \text{number of occurrences of pair } p \text{ in category } i \text{ after merges } 1, \dots, t - 1. \quad (3)$$

In plain English: this is just a scoreboard. For each category and each stage of BPE training, it tells you how often every neighboring pair appears.

If the true mixture weights were α , then the expected count of pair p at step t under that mixture is

$$s_p^{(t)} = \sum_{i=1}^n \alpha_i c_{i,p}^{(t)}. \quad (4)$$

In plain English: mix the category-specific counts using the unknown proportions. If English is 70% and code is 30%, then the pair count under the mixture is just the same weighted average.

Because BPE always picks the most frequent pair, the true merge $m^{(t)}$ must beat every other pair $p \neq m^{(t)}$:

$$\sum_{i=1}^n \alpha_i c_{i,m^{(t)}}^{(t)} \geq \sum_{i=1}^n \alpha_i c_{i,p}^{(t)} \quad \text{for all } t \in [M], p \neq m^{(t)}. \quad (5)$$

In plain English: at each merge step, the observed winner must really look like the winner under your guessed mixture. If your guessed mixture says some other pair should have won, then that guess is wrong.

That gives the ideal feasibility problem:

$$\text{find } \alpha \in \mathbb{R}^n \quad (6)$$

$$\text{s.t. } \sum_{i=1}^n \alpha_i c_{i,m^{(t)}}^{(t)} \geq \sum_{i=1}^n \alpha_i c_{i,p}^{(t)} \quad \forall t, \forall p \neq m^{(t)}, \quad (7)$$

$$\alpha_i \geq 0 \quad \forall i, \quad (8)$$

$$\sum_{i=1}^n \alpha_i = 1. \quad (9)$$

In plain English: find mixture weights that make every observed merge decision consistent. If you had perfect data and infinite samples, this would be the clean version.

But the attacker only has sampled corpora, so these constraints can conflict. The paper fixes that by adding slack variables $v^{(t)} \geq 0$ and $v_p \geq 0$:

$$v^{(t)} + v_p + \sum_{i=1}^n \alpha_i c_{i,m^{(t)}}^{(t)} \geq \sum_{i=1}^n \alpha_i c_{i,p}^{(t)} \quad \text{for all } t, p \neq m^{(t)}. \quad (10)$$

In plain English: if the counts do not line up perfectly, the model can pay a penalty and move on. The two slack terms absorb mismatch from noisy samples or missing categories.

The optimization problem, called LP1 in the paper, is

$$\min_{\alpha, \{v^{(t)}\}, \{v_p\}} \sum_{t=1}^M v^{(t)} + \sum_p v_p \quad (11)$$

$$\text{s.t. } v^{(t)} + v_p + \sum_{i=1}^n \alpha_i c_{i,m^{(t)}}^{(t)} \geq \sum_{i=1}^n \alpha_i c_{i,p}^{(t)} \quad \forall t, \forall p \neq m^{(t)}, \quad (12)$$

$$\alpha_i \geq 0 \quad \forall i, \quad (13)$$

$$v^{(t)} \geq 0 \quad \forall t, \quad (14)$$

$$v_p \geq 0 \quad \forall p, \quad (15)$$

$$\sum_{i=1}^n \alpha_i = 1. \quad (16)$$

In plain English: among all mixtures that roughly explain the merge list, choose the one that needs the least total excuse-making. Small total slack means your guessed mixture matches the tokenizer's behavior closely.

Because using all M merges is too expensive, they truncate to the first T merges. This gives LP1_T , which is the same problem but only for $t \leq T$.

$$\min_{\alpha, \{v^{(t)}\}_{t=1}^T, \{v_p\}} \sum_{t=1}^T v^{(t)} + \sum_p v_p \quad (17)$$

$$\text{s.t. } v^{(t)} + v_p + \sum_{i=1}^n \alpha_i c_{i,m^{(t)}}^{(t)} \geq \sum_{i=1}^n \alpha_i c_{i,p}^{(t)} \quad \forall t \leq T, \forall p \neq m^{(t)}, \quad (18)$$

$$\alpha_i \geq 0, v^{(t)} \geq 0, v_p \geq 0, \sum_i \alpha_i = 1. \quad (19)$$

In plain English: they only use the early part of the merge list, where the signal is strongest and the compute is manageable. This is a practical compromise, not a change in the core idea.

For fast violation checking, define

$$s_p^{(t)} = \sum_{i=1}^n \alpha_i c_{i,p}^{(t)}. \quad (20)$$

In plain English: this is the mixture-weighted score of pair p at step t . Once you have these scores, checking constraints becomes a max-search problem.

Then the constraints at step t are equivalent to

$$v^{(t)} + s_{m^{(t)}}^{(t)} \geq \max_p (s_p^{(t)} - v_p). \quad (21)$$

In plain English: the observed merge only needs to beat the strongest adjusted competitor. So instead of checking every pair one by one from scratch, you just need to know who the current top rival is.

The paper uses a priority queue, or heap (a data structure for quickly retrieving the largest or smallest item), to maintain the values $s_p^{(t)} - v_p$ across time. The key structural fact is that most counts do not change after a merge:

$$c_{i,p}^{(t+1)} = c_{i,p}^{(t)} \quad \text{unless pair } p \text{ overlaps with merge } m^{(t)}. \quad (22)$$

In plain English: one merge only disturbs nearby pairs. That means you do not need to recompute the whole world after every step — just the local neighborhood.

Their evaluation metric is mean squared error, or MSE (average squared difference between estimated and true mixture weights):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{\alpha}_i - \alpha_i^*)^2. \quad (23)$$

In plain English: if your estimate is close to the true mixture in every category, MSE is tiny. Squaring makes large mistakes hurt a lot more than small ones.

They report

$$\log_{10}(\text{MSE}). \quad (24)$$

In plain English: this compresses the scale so huge improvements are easy to see. A drop of 1 means a 10× lower MSE; a drop of 2 means a 100× lower MSE.

One useful way to read their results is by converting log-MSE gaps into error-ratio gaps. If method A has score a and method B has score b , then

$$\frac{\text{MSE}_A}{\text{MSE}_B} = 10^{a-b}. \quad (25)$$

In plain English: if their method gets -7.30 and a baseline gets -2.12 , then the gap is -5.18 , which means about $10^{5.18} \approx 1.5 \times 10^5$ times lower MSE. That is not a small win. That is a demolition.

There is no gradient derivation here because this is not trained with backpropagation. The optimization is a linear program solved directly. That is part of the appeal: the method is transparent, the constraints are interpretable, and every number has a concrete meaning tied to BPE training dynamics.

Does It Work?

The controlled experiments are strong and unusually clean. They train tokenizers themselves on known mixtures, so they know the ground truth exactly. Three settings: natural languages, programming languages, and English domains. The language pool is 112 languages from Oscar v23.01. The code pool is 37 programming languages from the GitHub split of RedPajama. The domain pool has 5 English sources: Wikipedia, web, books, code, and academic text. Each tokenizer is trained on 10 GB of mixed data with vocabulary size 30,000, using HuggingFace tokenizers and simple pretokenization: split on whitespace, and only merge digits with contiguous digits. For the attack, they estimate pair frequencies from 1 GB per category. Metric is $\log_{10}(\text{MSE})$ over 100 trials.

The headline result is blunt: their method wins by miles. For 5-way language mixtures, they get -7.30 ± 1.31 log-MSE, versus -2.02 for the tokenizer-efficiency baseline and -2.12 for the token-classification baseline. That is roughly 5 orders of magnitude better MSE than the baselines. For 10-way language mixtures, they get -7.66 versus -2.29 and -2.55 . For 30-way language mixtures, -7.73 versus -3.07 and -3.42 . Even at 112 languages, they still get -7.69 , while the best baseline is only -4.46 . Code mixtures are also strong: for 5-way code mixtures, -6.46 versus -2.54 and -1.92 . Domains are the hardest, which makes sense because books, Wikipedia, and web English share a lot of vocabulary. But even there, for 5-way domain mixtures, they get -3.74 versus -1.69 and -1.64 — still about 100× better MSE. Impressive because the signal is genuinely subtle there.

The commercial-tokenizer results are the fun part, and also the part to read with a slightly raised eyebrow. They infer GPT-2 is 99.1% English, GPT-3.5 is 62.6% code, GPT-4O is 39.0% non-English with 68 languages above 0.1%, LLAMA is basically restricted to Latin/Cyrillic-script languages, and LLAMA 3's extra 27,744 merges look like a multilingual-plus-code extension of GPT-3.5. These are plausible and often line up with public statements. The book estimates are the most provocative: they infer 7% to 26% books across all tokenizers they study. That is interesting, maybe important, and definitely not proven. The method is inferring tokenizer-training mixture, not full LM pretraining mixture, and the paper itself hints those may differ.

There are some gaps. First, robustness under distribution shift drops hard: when tokenizers are trained on web text but pair counts are estimated from Wikipedia in the same languages, log-MSE worsens from -7.66 to -3.53 . That is still much better than random, but it tells you the method depends on having reasonably matched reference corpora. Second, the paper does not fully solve the unknown-category problem. If the real tokenizer saw data outside your candidate set, the LP has to explain

that away with slack, and the estimates can bend. Third, character-level BPE, like SentencePiece with characters instead of bytes, is notably harder: average log-MSE drops to -4.09 instead of about -7.65 in comparable byte-level language experiments. So the method is excellent, but not magic. It works best when your candidate categories are well chosen and your corpora look like the tokenizer’s real diet.

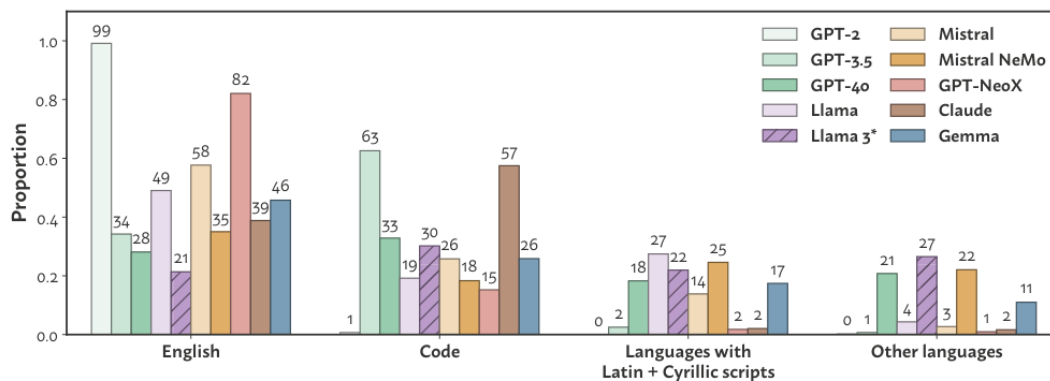


Figure 2: Training data mixture predictions for several commercial tokenizers. Complete results over 112 languages and 5 domains are given in §C; categories are grouped here for readability. We confirm that GPT-2 was trained overwhelmingly on English (99%), while GPT-3.5 is the first model in the GPT series to train on substantial code data (63%). GPT-4o is much more multilingual than its predecessors, with 39% of its corpus being non-English text. LLAMA is also multilingual, but focuses on languages using Latin or Cyrillic scripts (note this category in the figure excludes English). LLAMA 3* results are only based on the last 27,744 merges (the first 100K are copied from GPT-3.5), which we observe was primarily for multilingual adaptation.

This is the money figure for commercial tokenizers. The grouped bars make the big story obvious: GPT-2 is almost all English, GPT-3.5 jumps hard into code, GPT-4o becomes much more multilingual, and LLAMA stays concentrated on Latin/Cyrillic-script languages. Pay attention to LLAMA 3*: those results are only for the extra 27,744 merges after the copied GPT-3.5 block, which is why the multilingual adaptation story is so striking.

Blind Spots & Gotchas

categories:

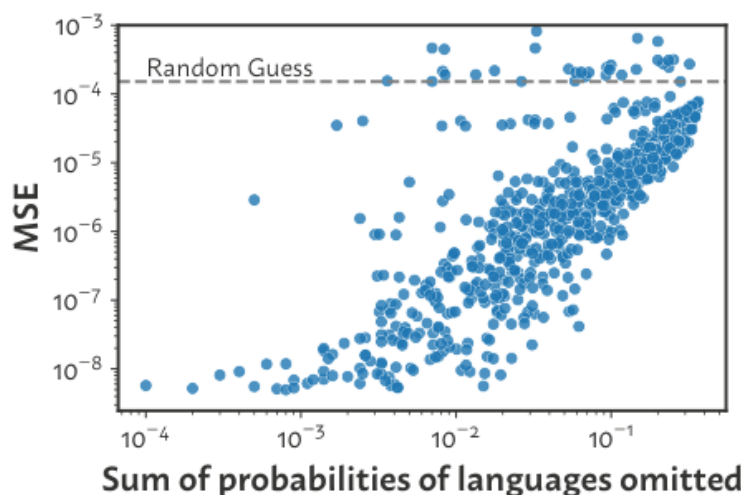


Figure 4: Performance remains much better than random even with large amounts of unknown data.

This is the robustness-to-missing-categories picture. Performance gets worse as more probability mass is assigned to omitted languages, which is exactly what you would expect. But the useful takeaway is that the method degrades gradually rather than collapsing immediately.

Blind Spots & Gotchas

- Implementation gotcha: if you recompute all pair counts from scratch after every merge, the method becomes unusably slow. The whole system depends on incremental updates, because only pairs overlapping the latest merge actually change.
- Implementation gotcha: the LP must enforce both $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$. If you forget the simplex constraint, the solver can scale weights arbitrarily and the inferred mixture becomes meaningless while still looking numerically 'good.'
- Implementation gotcha: lazy constraint generation only works if your violation checker is exact enough. If the priority-queue updates miss a changed pair, you can falsely declare the reduced LP optimal and get a wrong answer with no obvious warning.
- Paper blind spot: the attack assumes the true training data is well approximated by a mixture of the categories you supplied. If an important category is missing — say forum text, OCR junk, or multilingual code comments — the LP will force that mass into the nearest wrong bucket.
- Paper blind spot: strong distribution shift hurts a lot. They show web-trained tokenizers attacked with Wikipedia corpora degrade from -7.66 to -3.53 log-MSE. That is still useful, but it means the method is not a universal forensic tool unless your reference corpora are close to reality.

So What?

This paper matters because it turns a vague hunch into a precise attack. People already suspected tokenizers reveal something about training data. The paper shows you can recover actual mixture proportions with surprising accuracy, using only the public tokenizer and some reference corpora. That is a real shift. Tokenizers stop looking like harmless plumbing and start looking like audit artifacts. If a company ships a tokenizer, it may be shipping a compressed summary of what it cared about: English versus multilingual, code versus prose, books versus web, and even script coverage.

In practice, this opens several doors. For AI auditing, you can estimate whether a model family underrepresents certain languages or over-relies on books, code, or web crawl text. For security, it can guide membership-inference or data-source attacks by telling you where to look first. For NLP on filings, legal text, or scientific papers, it gives a way to check whether a tokenizer is a bad fit before you waste time fine-tuning a model on top of it. In quantitative finance, there is a neat angle too: if you build language models for earnings calls, SEC filings, broker notes, or news, tokenizer mixture inference could help you audit whether a foundation model's tokenizer was shaped more by code, web chatter, books, or formal documents. That matters for forecasting, risk signals, retrieval quality, and even execution systems that depend on robust text parsing under weird formatting.

A concrete extension: move from coarse categories to hierarchical inference. First infer top-level buckets like English, non-English, code, books, web, academic. Then, inside each bucket, infer sub-mixtures like specific languages, specific programming languages, or specific document types. You could also combine tokenizer evidence with model behavior, like perplexity or next-token probabilities, to get a stronger joint audit. My closing take: this is one of those papers that makes an ordinary object suddenly look dangerous. After reading it, you cannot see a public tokenizer as just a convenience file anymore.